#Ite 1

Conflict & Ambiguity

7. Root Cause Analysis
- Error Cause: The issue likely involves incorrect handling of device pixel ratio on retina displays during canvas reinitialization after unpickling. The DPI may be multiplied by device_pixel_ratio multiple times.
- Code Paths:
  1. `figure.py` `__setstate__()` method (line 3035) restores figure state
  2. Creates new canvas via `FigureCanvasBase(self)` (line 3047)
  3. If restoring to pyplot, calls `plt._backend_mod.new_figure_manager_given_figure()`
  4. `backend_bases.py` `new_figure_manager_given_figure()` creates manager via canvas class
  5. `backend_macosx.py` `FigureCanvasMac.__init__()` calls `_macosx.FigureCanvas.__init__()`
  6. `_macosx.m` `FigureCanvas_init()` calls `get_width_height()` which uses DPI and device_pixel_ratio
- Key Functions: `get_width_height()` in `backend_bases.py`, device pixel ratio handling, DPI persistence during state restoration.

PassRatio=0%

#Ite 2

Conflict & Ambiguity

7. Root Cause Analysis
- Error Cause: During figure unpickling, `FigureCanvasBase.__init__` unconditionally sets `figure._original_dpi = figure.dpi`. When a figure from a retina display (device_pixel_ratio=2) is unpickled, its `figure.dpi` is already scaled (e.g., 200 instead of base 100). This overwrites `_original_dpi` with the scaled value. Later, when `_set_device_pixel_ratio(2)` is called, it calculates `dpi = 2 * figure._original_dpi = 2 * 200 = 400`, causing double scaling. The `_original_dpi` attribute should represent the base DPI (without device_pixel_ratio scaling) but becomes incorrectly set to the scaled DPI.
- Code Paths:
  1. `figure.py` `__setstate__()` method (line 3035) restores figure state
  2. Creates new canvas via `FigureCanvasBase(self)` (line 3047)
  3. `backend_bases.py` `FigureCanvasBase.__init__()` (line 1656) sets `figure._original_dpi = figure.dpi`
  4. If restoring to pyplot, calls `plt._backend_mod.new_figure_manager_given_figure()`
  5. `backend_bases.py` `new_figure_manager_given_figure()` creates manager via canvas class
  6. `backend_macosx.py` `FigureCanvasMac.__init__()` calls parent inits
  7. `_macosx.m` C code calls `updateDevicePixelRatio()` which calls `_set_device_pixel_ratio(2)`
  8. `backend_bases.py` `_set_device_pixel_ratio()` (line 2104) calculates `dpi = ratio * figure._original_dpi`
- Key Functions: `FigureCanvasBase.__init__()` in `backend_bases.py`, `_set_device_pixel_ratio()` in `backend_bases.py`, device pixel ratio handling.

PassRatio=60%

#Ite 3

Ambiguity

7. Root Cause Analysis
  - Error Cause: During figure unpickling, `Figure.__setstate__()` calls `FigureCanvasBase(self)` to reinitialize the canvas. In `FigureCanvasBase.__init__()` at line 1656, the code unconditionally sets `figure._original_dpi = figure.dpi`. When a figure from a retina display (device_pixel_ratio=2) is unpickled, its `figure.dpi` is already the scaled value (e.g., 200 instead of base 100). This overwrites `_original_dpi` with the scaled value, losing the original base DPI from the pickled state. Later, when the macOS backend calls `_set_device_pixel_ratio(2)` during canvas initialization, it calculates `dpi = 2 * figure._original_dpi = 2 * 200 = 400`, causing double scaling. The `_original_dpi` attribute should represent the base DPI (without device_pixel_ratio scaling) but becomes incorrectly set to the scaled DPI.
  - Code Paths:
  1. `figure.py` `__setstate__()` method (line 3035) restores figure state from pickle
  2. Calls `FigureCanvasBase(self)` (line 3047) to recreate canvas
  3. `backend_bases.py` `FigureCanvasBase.__init__()` (line 1656) sets `figure._original_dpi = figure.dpi`, overwriting pickled value
  4. If restoring to pyplot, calls `plt._backend_mod.new_figure_manager_given_figure()`
  5. `backend_bases.py` `new_figure_manager_given_figure()` creates manager via canvas class
  6. `backend_macosx.py` `FigureCanvasMac.__init__()` calls parent initializations
  7. `_macosx` C extension code calls `updateDevicePixelRatio()` which triggers `_set_device_pixel_ratio(2)`
  8. `backend_bases.py` `_set_device_pixel_ratio()` (line 2104) calculates `dpi = ratio * figure._original_dpi` using incorrectly doubled `_original_dpi`
  - Key Functions: `FigureCanvasBase.__init__()` in `backend_bases.py`, `_set_device_pixel_ratio()` in `backend_bases.py`, device pixel ratio handling in macOS backend.

PassRatio=70%

#Ite 4

7. Root Cause Analysis
- Error Cause: During figure unpickling, `Figure.__setstate__()` calls `FigureCanvasBase(self)` to reinitialize the canvas. In `FigureCanvasBase.__init__()` at line 1656, the code unconditionally sets `figure._original_dpi = figure.dpi`. This causes a cumulative doubling effect:
  1. On a retina display (device_pixel_ratio=2), when a figure is first created, `figure.dpi` is scaled to 200, and `figure._original_dpi` is set to the base value 100.
  2. When pickled, both `figure.dpi=200` and `figure._original_dpi=100` are saved.
  3. When unpickled, `figure.dpi=200` and `figure._original_dpi=100` are restored.
  4. `FigureCanvasBase.__init__()` overwrites `figure._original_dpi` with `figure.dpi` (200), losing the original base value.
  5. Later, `_set_device_pixel_ratio(2)` is called, calculating `dpi = 2 * figure._original_dpi = 2 * 200 = 400`.
  6. This doubled `figure.dpi=400` and overwritten `figure._original_dpi=200` are saved in the next pickle cycle.
  The core issue is that `figure._original_dpi` should represent the base DPI (without device pixel ratio scaling) but becomes incorrectly overwritten with the already-scaled `figure.dpi` value during canvas reinitialization.
- Code Paths:
  1. `figure.py` `__setstate__()` method (line 3035) restores figure state from pickle
  2. Calls `FigureCanvasBase(self)` (line 3047) to recreate canvas
  3. `backend_bases.py` `FigureCanvasBase.__init__()` (line 1656) sets `figure._original_dpi = figure.dpi`, overwriting pickled value
  4. If restoring to pyplot, calls `plt._backend_mod.new_figure_manager_given_figure()`
  5. `backend_bases.py` `new_figure_manager_given_figure()` creates manager via canvas class
  6. `backend_macosx.py` `FigureCanvasMac.__init__()` calls parent initializations
  7. `_macosx` C extension code calls `updateDevicePixelRatio()` which triggers `_set_device_pixel_ratio(2)`
  8. `backend_bases.py` `_set_device_pixel_ratio()` (line 2104) calculates `dpi = ratio * figure._original_dpi` using incorrectly doubled `_original_dpi`
- Key Functions: `FigureCanvasBase.__init__()` in `backend_bases.py`, `_set_device_pixel_ratio()` in `backend_bases.py`, device pixel ratio handling in macOS backend.

PassRatio=100%